AI Assisted Coding

Week7 - Wednesday

Name: pruthagwin challuri

Roll No.: 2303A51951

Batch No.: 07

Assignment Number: 13.3

Lab 13: Code Refactoring Using AI Assistance Improving Legacy Code for Readability, Maintainability, and Performance

---

🔹 Task 1: Removing Code Duplication

---

1) Prompt to Give in VS Code

I have a legacy Python script that calculates area and perimeter of rectangles using repeated print statements and hardcoded values.

Please refactor the code by:
- Removing duplicated logic
- Creating a reusable function
- Adding proper docstrings
- Keeping the same output behavior
- Following clean Python best practices

---

2) Refactored Code

```
def calculate_rectangle(length, width):
    """
    Calculate area and perimeter of a rectangle.

    Parameters:
```

```
    length (int or float): Length of the rectangle
    width (int or float): Width of the rectangle

  Returns:
    tuple: (area, perimeter)
  """
  area = length * width
  perimeter = 2 * (length + width)
  return area, perimeter


rectangles = [(5, 10), (7, 12), (10, 15)]

for length, width in rectangles:
  area, perimeter = calculate_rectangle(length, width)
  print("Area of Rectangle:", area)
  print("Perimeter of Rectangle:", perimeter)
```

---

3) Output

```
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
```

---

4) Remarks

✓ Removed repeated logic
✓ Improved maintainability
✓ Easy to reuse for future rectangles
✓ Cleaner and modular structure

🔹 Task 2: Optimizing Loops and Conditionals

---

1) Prompt

Refactor the following nested loop code that checks if names exist in a list.

Replace inefficient nested loops with optimized structures like sets.
Provide performance comparison explanation.
Keep same output.

---

2) Refactored Code

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

# Convert list to set for O(1) lookup
names_set = set(names)

for s in search_names:
  if s in names_set:
    print(f"{s} is in the list")
  else:
    print(f"{s} is not in the list")
```

---

3) Output

```
Charlie is in the list
Eve is not in the list
Bob is in the list
```

---

4) Remarks

Before Refactoring:

Nested loop → Time Complexity: **O(n × m)**

After Refactoring:

Set lookup → Time Complexity: **O(n + m)**

✓ Faster for large datasets
✓ Cleaner logic
✓ Improved readability

🔹 Task 3: Extracting Reusable Functions

---

1) Prompt

Refactor legacy inline tax calculation code.
Create a reusable function calculate_total(price).
Add proper documentation and maintain identical output.

---

2) Refactored Code

```
def calculate_total(price):
    """
    Calculate total price including 18% tax.

    Parameters:
        price (int or float): Base price

    Returns:
        float: Total price including tax
    """
    tax_rate = 0.18
    tax = price * tax_rate
    return price + tax


prices = [250, 500]

for price in prices:
    total = calculate_total(price)
    print("Total Price:", total)
```

---

3) Output

```
Total Price: 295.0
Total Price: 590.0
```

---

4) Remarks

✓ Removed duplicate logic
✓ Improved modularity
✓ Easy to change tax rate in one place

🔹 Task 4: Replacing Hardcoded Values with Constants

---

1) Prompt

Refactor code that calculates area and circumference of a circle.
Replace magic numbers like 3.14159 and 7 with named constants.
Improve readability and maintainability.

---

2) Refactored Code

```
PI = 3.14159
RADIUS = 7

area = PI * (RADIUS ** 2)
circumference = 2 * PI * RADIUS

print("Area of Circle:", area)
print("Circumference of Circle:", circumference)
```

---

3) Output

```
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

---

4) Remarks

✓ Eliminated magic numbers
✓ Easier to modify radius or PI
✓ Improved readability

🔹 Task 5: Improving Variable Naming and Readability

---

1) Prompt

Refactor a script with unclear variable names.
Replace variables a, b, c with meaningful names.
Add inline comments.
Keep same output.

---

2) Refactored Code

```
base = 10
height = 20

# Calculate area of triangle
triangle_area = base * height / 2

print(triangle_area)
```

---

3) Output

```
100.0
```

---

4) Remarks

✓ Improved clarity
✓ Meaningful variable names
✓ Easier for new developers to understand

✅ **Final Overall Remarks for Lab 13**

AI-Assisted Refactoring Helps:

✓ Detect duplicate logic
✓ Suggest optimized data structures
✓ Improve naming conventions
✓ Enhance documentation quality
✓ Improve performance